



Université de Sherbrooke

SQL

Modification des données

SQL_04

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

Ameni SOUID (ameni.Souid@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

—

Scriptorum/Scriptorum/SQL_04-Modification, version 1.0.0.a, en date du 2026-02-03

— document de travail, ne pas citer —

Sommaire

...

Mise en garde

Le présent document est en cours d'élaboration ; en conséquence, il est incomplet et peut contenir des erreurs.

Historique

diffusion	resp.	description
2026-02-10	CK	Récupération de notes diverses.

Table des matières

Introduction.....	4
1. Présentation.....	4
2. Insertion.....	4
3. Retrait.....	5
4. Mise à jour.....	5
5. Comportement des clés référentielles	6
Conclusion.....	9
Références	10
Définitions	11

Introduction

Le présent document a pour but de présenter une introduction au langage de manipulation de données qu'on retrouve dans SQL.

Dans un premier temps, nous présenterons une grammaire simplifiée, sous-ensemble propre de la grammaire de la norme ISO 9075:2016, avec, au besoin, certaines particularités de PostgreSQL.

1. Présentation

- Insertion de données
- Retrait de données
- Mise à jour de données

2. Insertion

```
insertion ::=
  [ contexte ]
  INSERT INTO nom_table
  [ ( listeNomsColonne ) ]
  { DEFAULT VALUES | VALUES listeExpressionInit | requête }
  [ suiteInsertion ]

listeExpressionInit ::=
  { tuple-val ... , }

tuple-val ::=
  ( { { expression | DEFAULT } ... , } )
```

Il existe plusieurs méthodes d'ajout :

- ajout d'un seul tuple ;
- ajout d'un ensemble de tuples à partir de valeurs explicitement déclarées ;
- ajout d'un ensemble de tuples à partir d'une requête ;
- ajout d'un ensemble de tuples à partir d'un fichier (ex. csv, json) ;

Dans tous les cas, est fortement recommandé de toujours donner explicitement la liste des colonnes.

Exemple — Université

- Ajouter le type d'évaluation Travail dirigé.

```
insert into TypeEvaluation (code, description)
values ('TD', 'Travail dirigé');
```

- Ajouter une nouvelle activité au programme d'informatique.

```
insert into Activite (sigle, titre)
values ('IFT339', 'Structure de données');
```

- Ajouter des résultats.

```
-- /!\ Omission de la liste des colonnes
insert into Resultat values
('15113150', '20133', 'IFT187', 'TP', 80);
```

```
insert into Resultat (matricule, activite, trimestre, TE, note)
values ('15113150', 'IFT187', '20133', 'TP', 80);

insert into Resultat (matricule, activité, trimestre, TE, note)
values
('15112354', 'IFT187', '20123', 'FI', 78),
('15113150', 'IFT159', '20133', 'TP', 75),
('15112354', 'GMQ103', '20123', 'FI', 85),
('15110132', 'IMN117', '20123', 'IN', 90),
('15110132', 'IFT187', '20133', 'IN', 45),
('15112354', 'IFT159', '20123', 'FI', 52);
```

3. Retrait

```
retrait ::=
  [ contexte ]
  DELETE FROM [ONLY] nomTable [ * ]
  [ [ AS ] alias ]
  [ USING liste_from ]
  [ WHERE condition ]
  [ suiteRetrait ]
```

La clause USING a un rôle analogue au FROM du select sauf que :

- elle est limitée aux seuls produits cartésiens (pas de jointure) ;
- la table nomTable en fait déjà partie et sera jointe à la liste_from.

Corolairement, si on désire en faire une jointure, il faut l'exprimer par une condition au sein de la clause WHERE.

Exemple — Université

- Retirer le type d'évaluation TP

```
delete from TypeEvaluation
where code = 'TP';
```

- Retirer les notes des TP.

```
delete from Resultat
where TE = 'TP';
```

- Retirer les activités IMN du catalogue.

```
delete from Activite
where substr(sigle,1,3) = 'IMN';
```

4. Mise à jour

```
mise_à_jour ::=
```

```

[ contexte ]
UPDATE [ ONLY ] nom_table [ * ] [ [ AS ] alias ]
SET { paire , ... }
[ FROM liste_from ]
[ WHERE condition ]
[ suite_mise_à_jour ]

paire ::=
  nomCol = val
  | ( { nomCol ... , } ) = ( { val ... , } )

val ::= { expression | DEFAULT }

```

La clause FROM de UPDATE a un rôle analogue au FROM du SELECT sauf que :

- elle est limitée aux seuls produits cartésiens (pas de jointure) ;
- la table nomTable en fait déjà partie et sera jointe à la liste_from.

Corolairement, si on désire en fait une jointure, il faut l'exprimer par une condition au sein de la clause WHERE.

Exemple — Université

- Ajouter 10% pour tous les étudiants.

```

update Resultat
  set note = note + (note * 0,1);

```

- Retirer 5 points à la note des TP de tous les étudiants de l'activité 'IFT187'.

```

update Resultat
  set note = note - 5
where TE = 'TP'
  and activite = 'IFT187';

```

- Retirer 5 points à Éliane pour son cours IFT187.

```

update Resultat
  set note = note - 5
from Etudiant
where nom = 'Éliane'
  and etudiant.matricule = resultat.matricule
  and activite = 'IFT187';

```

/?\ Que se passe-t-il si Éliane a suivi le cours deux fois en faisant chaque fois l'examen final ?

5. Comportement des clés référentielles

Que peut-il se passer lors de la modification d'un attribut participant à une clé référentielle ?

Que se passe-t-il quand

- un tuple référent est détruit ?
- un tuple référent est modifié ?
- un tuple référent est détruit ?
- un tuple référent est modifié ?

Lors de la définition des contraintes il est possible de définir des actions à prendre.

```
foreignKey ::=
  FOREIGN KEY ( listeNomsColonne ) REFERENCES nomTable [ ( listeNomsColonne ) ]
  [ MATCH { SIMPLE | PARTIAL | FULL } ]
  [ ON UPDATE action ]
  [ ON DELETE action ]

action ::=
  NO ACTION | CASCADE | SET NULL | SET DEFAULT
```

Le terme [MATCH { SIMPLE | PARTIAL | FULL }] permet de contrôler le traitement des NULL lors de la comparaison des valeurs pour un ensemble d'attributs référencés :

- SIMPLE (par défaut), n'importe quel attribut peut avoir une valeur nulle, si l'une d'entre elles est nulle, les autres valeurs ne sont pas tenues d'avoir une correspondance dans la table référencée.
- FULL, toutes les valeurs doivent avoir une correspondance ou toutes les valeurs doivent être nulles.
- PARTIAL, au moins un des attributs doit avoir une correspondance (les autres valeurs peuvent être nulles). /!\ Non disponible dans PostgreSQL.

Les termes [ON UPDATE action] et [ON DELETE action] permettent de spécifier une action lorsqu'une valeur d'un clé référentielle est modifiée.

- NO ACTION (par défaut), la suppression et la mise à jour dans la relation référencée s'effectuent en respectant la contrainte de clé référentielle.
- CASCADE, spécifie que lorsqu'un tuple référencé est supprimé, les tuples qui le référencent doivent également être supprimés.
- SET NULL (non applicable pour la mise à jour), spécifie que lorsqu'un tuple référencé est supprimé, les valeurs des attributs qui le référencent deviennent nulles.
- SET DEFAULT (non applicable pour la mise à jour) spécifie que lorsqu'un tuple référencé est supprimé, les valeurs des attributs qui le référencent prennent les valeurs spécifiées. Si la valeur par défaut ne satisfait pas la contrainte de clé référentielle, la transaction échouera.

Dans notre exemple, nous avons trois clés référentielles où la relation Resultat est la **relation référence** et les relations Etudiant, TypeEvaluation et Activite sont les **relations référencées**.

Rappelons qu'une contrainte de clé référentielle exige que les valeurs d'un attribut (ou d'un ensemble d'attributs) soient identiques à un tuple d'une autre relation. On dit que cela maintient l'intégrité référentielle entre les deux tables.

Le fait que les attributs référés forment une clé de la table référée simplifie grandement la description du comportement et sa mise en oeuvre; aussi SQL en a-t-il fait une obligation.

Au fait, pourquoi le cas de l'insertion n'est-il pas traité ?

- Une insertion dans la table référée n'induit aucun problème du côté des tables référentes.
- Dans le cas général, une insertion d'une clé non référée dans une table référente, ne peut être « compensée » automatiquement.
- Dans les cas particuliers où cela est possible, un automatisme spécifique peut être défini, ce que nous étudierons ultérieurement.

UNIVERSITÉ

Étudiant clé (matricule)

matricule	nom	adresse
15113150	Paul	>Δ ^ς σ ^ς ᵇ
15112354	Éliane	Blanc-Sablon
15113870	Mohamed	Tadoussac
15110132	Sergeï	Chandler

Activité clé (sigle)

sigle	titre
IFT159	Analyse et programmation
IFT187	Éléments de bases de données
IMN117	Acquisition des médias numériques
IGE401	Gestion de projets
GMQ103	Géopositionnement

TypeEvaluation clé(code)

code	description
IN	Examen intra
FI	Examen final
TP	Travail pratique
PR	Projet

Résultat clé(matricule, te, activite, trimestre)

matricule	TE	activité	trimestre	note
15113150	TP	IFT187	20132	80
15112354	FI	IFT187	20122	78
15113150	TP	IFT159	20132	75
15112354	FI	GMQ103	20122	85
15110132	IN	IMN117	20123	90
15110132	IN	IFT187	20133	45
15112354	FI	IFT159	20123	52

Figure 1. Exemple — Université

Conclusion

Références

[Elmasri2016]

Ramez ELMASRI et Shamkant B. NAVATHE;
Fundamentals of database systems;
7th Edition, Pearson, Hoboken (NJ, US), 2016;
ISBN 978-0-13-397077-7.

[Date2015a]

Chris J. DATE;
SQL and relational theory: how to write accurate SQL code;
O'Reilly Media, 2015;
ISBN 978-1-4919-4117-1.

PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-09-16 16:46:13 UTC



Université de Sherbrooke